



UNIVERSIDAD
NACIONAL
DE COLOMBIA

Project documentation - RabbitCTF

Equipo Alfa Buena Maravilla Onda Dinamita Escuadrón Lobo

Juan David Buitrago Salazar – jbuitragosa@unal.edu.co

Luis David Garzón Morales – lgarzonmo@unal.edu.co

Juan David Cárdenas Galvis – jcardenasgal@unal.edu.co

Álvaro Camilo Torres Rodríguez – alvtorresro@unal.edu.co

David Felipe Chaparro Perez – dachaparro@unal.edu.co

Professor

Liliana Marcela Olarte Mesa

Universidad Nacional de Colombia

Department of Systems and Industrial Engineering

Software Engineering II

Bogotá D.C., Colombia

2025

1. Database Script

Rabbit CTF involves a previous design of a database model or schema in a UML diagram. For this task the team used the database design tool dbdiagram.io which allows create each entity or table with their attributes and relations between these by coding instructions defined by the tool.

The database model designed can be looked in the next link:

<https://dbdiagram.io/d/RabbitCTF-Project-Database-693602fa3c4ea889c6e43631>

With the model created, and considering that for this app deployment was used Docker and Docker-compose, an SQL script was written for creation the database schema designed with the name `'01_create_db.sql'` in the project path `'RabbitCTF/backend/db/init'`. Then, other two SQL scripts were created for inserting important initial data for testing all functionalities of the app with the name `'02_data_seed.sql'` and `'02_seed_data_minimal.sql.bak'` in the project path as the previous script.

SQL script for Database creation

This script establishes the base schema for the RabbitCTF platform using PostgreSQL. It is critical to the business logic and enforces the rules of the competition: ensuring users belong to teams, challenges have specific scoring configurations (static or. decay), and preventing duplicate flag submissions for points. It also manages the event lifecycle through configuration and audit logging.

For the creation of each table or entity and its attributes is used the SQL syntax defined by:

```
CREATE TABLE IF NOT EXISTS <name_table> (...)
```

This instruction guarantees that the table are created only if don't exist yet. Then in the body are defined all attributes of that entity of table ensuring data integrity, strict typing, but more important, defining the bases or characteristics for applying business rules. Each of these attributes have a specific type, default values and also can have constraints.

Moreover, relationships between tables are defined with the instruction *FOREIGN KEY (attribute) REFERENCES <referenced_table>(referenced_attribute)* with created a foreign key. Foreign keys are essential for referential integrity, linking entities like Users to Roles, or Submissions to Challenges. This prevents orphan data, as for example a submission belonging to a non-existent team

```
-- RabbitCTF Database Schema - MVP Complete Version
-- Para uso con Docker Compose

-- =====
-- REFERENCE DATA (ENUMS AS TABLES)
```

```
-- =====  
  
CREATE TABLE IF NOT EXISTS role (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(30) UNIQUE NOT NULL,  
    description TEXT,  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
CREATE TABLE IF NOT EXISTS difficulty (  
    id SERIAL PRIMARY KEY,  
    name VARCHAR(20) UNIQUE NOT NULL,  
    sort_order INT NOT NULL,  
    description TEXT,  
    created_at TIMESTAMP DEFAULT NOW()  
);  
  
-- =====  
-- USER MANAGEMENT  
-- =====  
  
CREATE TABLE IF NOT EXISTS "user" (  
    id SERIAL PRIMARY KEY,  
    username VARCHAR(50) UNIQUE NOT NULL,  
    email VARCHAR(100) UNIQUE NOT NULL,  
    role_id INT NOT NULL,  
    created_at TIMESTAMP DEFAULT NOW(),  
    updated_at TIMESTAMP,  
    FOREIGN KEY (role_id) REFERENCES role(id)  
);  
  
CREATE TABLE IF NOT EXISTS user_credential (  
    user_id INT PRIMARY KEY,  
    password_hash VARCHAR(255) NOT NULL,  
    is_temp_password BOOLEAN DEFAULT FALSE,  
    must_change_password BOOLEAN DEFAULT FALSE,  
    last_changed TIMESTAMP DEFAULT NOW(),  
    FOREIGN KEY (user_id) REFERENCES "user"(id) ON DELETE CASCADE  
);  
  
CREATE TABLE IF NOT EXISTS password_reset_request (  
    id SERIAL PRIMARY KEY,  
    user_id INT NOT NULL,  
    requested_at TIMESTAMP DEFAULT NOW(),  
    is_processed BOOLEAN DEFAULT FALSE,  
    FOREIGN KEY (user_id) REFERENCES "user"(id)  
);
```

```
-- =====
-- TEAMS & MEMBERSHIP
-- =====

CREATE TABLE IF NOT EXISTS team (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL,
    captain_id INT UNIQUE NOT NULL,
    total_score INT DEFAULT 0,
    created_at TIMESTAMP DEFAULT NOW(),
    FOREIGN KEY (captain_id) REFERENCES "user"(id)
);

CREATE TABLE IF NOT EXISTS team_credential (
    team_id INT PRIMARY KEY,
    password_hash VARCHAR(255) NOT NULL,
    FOREIGN KEY (team_id) REFERENCES team(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS team_member (
    user_id INT PRIMARY KEY,
    team_id INT NOT NULL,
    joined_at TIMESTAMP DEFAULT NOW(),
    FOREIGN KEY (user_id) REFERENCES "user"(id),
    FOREIGN KEY (team_id) REFERENCES team(id) ON DELETE CASCADE
);

-- =====
-- CHALLENGES & CATEGORIES
-- =====

CREATE TABLE IF NOT EXISTS challenge_category (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) UNIQUE NOT NULL,
    description TEXT,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS challenge (
    id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    description TEXT NOT NULL,
    category_id INT,
    difficulty_id INT NOT NULL,
    created_by INT,
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP,
```

```
operational_data TEXT,  
is_draft BOOLEAN DEFAULT TRUE,  
FOREIGN KEY (category_id) REFERENCES challenge_category(id),  
FOREIGN KEY (difficulty_id) REFERENCES difficulty(id),  
FOREIGN KEY (created_by) REFERENCES "user"(id)  
);  
  
CREATE TABLE IF NOT EXISTS challenge_score_config (  
    challenge_id INT PRIMARY KEY,  
    scoring_mode VARCHAR(20) DEFAULT 'STATIC' NOT NULL,  
    base_score INT NOT NULL,  
    decay_factor FLOAT,  
    min_score INT,  
    FOREIGN KEY (challenge_id) REFERENCES challenge(id) ON DELETE CASCADE  
);  
  
CREATE TABLE IF NOT EXISTS challenge_rule_config (  
    challenge_id INT PRIMARY KEY,  
    attempt_limit INT DEFAULT 5,  
    is_case_sensitive BOOLEAN DEFAULT TRUE,  
    FOREIGN KEY (challenge_id) REFERENCES challenge(id) ON DELETE CASCADE  
);  
  
CREATE TABLE IF NOT EXISTS challenge_flag (  
    challenge_id INT PRIMARY KEY,  
    flag_value VARCHAR(255) NOT NULL,  
    FOREIGN KEY (challenge_id) REFERENCES challenge(id) ON DELETE CASCADE  
);  
  
CREATE TABLE IF NOT EXISTS challenge_visibility_config (  
    challenge_id INT PRIMARY KEY,  
    is_visible BOOLEAN DEFAULT FALSE,  
    visible_from TIMESTAMP,  
    visible_until TIMESTAMP,  
    FOREIGN KEY (challenge_id) REFERENCES challenge(id) ON DELETE CASCADE  
);  
  
CREATE TABLE IF NOT EXISTS challenge_file (  
    id SERIAL PRIMARY KEY,  
    challenge_id INT NOT NULL,  
    file_path VARCHAR(255) NOT NULL,  
    file_name VARCHAR(255) NOT NULL,  
    file_type VARCHAR(100),  
    file_size_mb FLOAT,  
    uploaded_at TIMESTAMP DEFAULT NOW(),  
    FOREIGN KEY (challenge_id) REFERENCES challenge(id) ON DELETE CASCADE  
);
```

```
-- =====
-- PARTICIPATION & FLAG SUBMISSIONS
-- =====

CREATE TABLE IF NOT EXISTS submission (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    team_id INT NOT NULL,
    challenge_id INT NOT NULL,
    submitted_flag VARCHAR(255) NOT NULL,
    is_correct BOOLEAN DEFAULT FALSE,
    awarded_score INT,
    submitted_at TIMESTAMP DEFAULT NOW(),
    FOREIGN KEY (user_id) REFERENCES "user"(id),
    FOREIGN KEY (team_id) REFERENCES team(id),
    FOREIGN KEY (challenge_id) REFERENCES challenge(id)
);

CREATE TABLE IF NOT EXISTS submission_block (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    challenge_id INT,
    blocked_until TIMESTAMP NOT NULL,
    reason VARCHAR(100),
    created_at TIMESTAMP DEFAULT NOW(),
    FOREIGN KEY (user_id) REFERENCES "user"(id),
    FOREIGN KEY (challenge_id) REFERENCES challenge(id)
);

-- =====
-- EVENT CONFIGURATION
-- =====

CREATE TABLE IF NOT EXISTS event_config (
    id SERIAL PRIMARY KEY,
    event_name VARCHAR(100) NOT NULL,
    start_time TIMESTAMP,
    end_time TIMESTAMP,
    status VARCHAR(20) DEFAULT 'not_started',
    max_team_size INT DEFAULT 4,
    max_submission_attempts INT DEFAULT 5,
    submission_time_window_seconds INT DEFAULT 60,
    submission_block_minutes INT DEFAULT 5,
    max_file_size_mb FLOAT DEFAULT 100,
    max_challenge_files_mb FLOAT DEFAULT 500,
    allowed_file_types JSON DEFAULT '["zip", "tar.gz", "txt", "pdf", "pcap",
"png", "jpg"]',
    discord_webhook_url VARCHAR(255),
```

```
    discord_bot_token_encrypted VARCHAR(255),
    discord_notifications_enabled BOOLEAN DEFAULT FALSE,
    allow_solution_history BOOLEAN DEFAULT FALSE,
    event_timezone VARCHAR(50) DEFAULT 'UTC',
    created_at TIMESTAMP DEFAULT NOW(),
    updated_at TIMESTAMP
);

-- =====
-- EVENT RULES
-- =====

CREATE TABLE IF NOT EXISTS event_rule_version (
    id SERIAL PRIMARY KEY,
    content_md TEXT NOT NULL,
    version_number INT NOT NULL,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW(),
    FOREIGN KEY (created_by) REFERENCES "user"(id)
);

CREATE TABLE IF NOT EXISTS event_rule_current (
    id INT PRIMARY KEY DEFAULT 1,
    active_version_id INT UNIQUE NOT NULL,
    updated_at TIMESTAMP DEFAULT NOW(),
    FOREIGN KEY (active_version_id) REFERENCES event_rule_version(id)
);

-- =====
-- NOTIFICATIONS
-- =====

CREATE TABLE IF NOT EXISTS notification (
    id SERIAL PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    message TEXT NOT NULL,
    type VARCHAR(30) NOT NULL,
    is_published BOOLEAN DEFAULT TRUE,
    created_by INT NOT NULL,
    created_at TIMESTAMP DEFAULT NOW(),
    published_at TIMESTAMP,
    FOREIGN KEY (created_by) REFERENCES "user"(id)
);

-- =====
-- AUDIT LOGGING
-- =====
```

```
CREATE TABLE IF NOT EXISTS audit_log (  
  id SERIAL PRIMARY KEY,  
  user_id INT,  
  action VARCHAR(50) NOT NULL,  
  resource_type VARCHAR(30),  
  resource_id INT,  
  details JSON,  
  ip_address VARCHAR(45),  
  created_at TIMESTAMP DEFAULT NOW(),  
  FOREIGN KEY (user_id) REFERENCES "user"(id)  
);
```

After defining all SQL instructions to create all required entities or tables based on the initial database model, indexes are created on required tables to enforce data integrity, avoiding duplicated data and improving query performance. Indexes are important for high-traffic queries, for example to calculate submissions score and team total score after validating that a flag submitted for a challenge is correct. The syntax of instructions to define indexes is:

```
CREATE INDEX IF NOT EXISTS <index_name> ON  
<table_name>(<referenced_attribute>);
```

It created an index over a specified attribute or column improving query performance over this column. In RabbitCTF, indexes prevents a team of getting points for the same challenge.

```
-- =====  
-- CRITICAL INDEXES FOR PERFORMANCE  
-- =====  
  
-- User indexes  
CREATE INDEX IF NOT EXISTS idx_user_username ON "user"(username);  
CREATE INDEX IF NOT EXISTS idx_user_email ON "user"(email);  
CREATE INDEX IF NOT EXISTS idx_user_role ON "user"(role_id);  
CREATE INDEX IF NOT EXISTS idx_user_created_at ON "user"(created_at);  
  
-- Team indexes  
CREATE INDEX IF NOT EXISTS idx_team_name ON team(name);  
CREATE INDEX IF NOT EXISTS idx_team_captain ON team(captain_id);  
CREATE INDEX IF NOT EXISTS idx_team_score ON team(total_score DESC);  
CREATE INDEX IF NOT EXISTS idx_team_created_at ON team(created_at);  
  
-- Team member indexes  
CREATE INDEX IF NOT EXISTS idx_team_member_team ON team_member(team_id);  
CREATE INDEX IF NOT EXISTS idx_team_member_joined ON team_member(joined_at);  
  
-- Challenge indexes
```



```
CREATE INDEX IF NOT EXISTS idx_challenge_category ON challenge(category_id);
CREATE INDEX IF NOT EXISTS idx_challenge_difficulty ON
challenge(difficulty_id);
CREATE INDEX IF NOT EXISTS idx_challenge_created_by ON
challenge(created_by);
CREATE INDEX IF NOT EXISTS idx_challenge_visible ON challenge(is_draft,
category_id, difficulty_id);
CREATE INDEX IF NOT EXISTS idx_challenge_created_at ON
challenge(created_at);

-- Challenge file indexes
CREATE INDEX IF NOT EXISTS idx_challenge_file_challenge ON
challenge_file(challenge_id);
CREATE INDEX IF NOT EXISTS idx_challenge_file_uploaded ON
challenge_file(uploaded_at);

-- Submission indexes (CRITICAL for performance)
CREATE INDEX IF NOT EXISTS idx_submission_user ON submission(user_id);
CREATE INDEX IF NOT EXISTS idx_submission_team ON submission(team_id);
CREATE INDEX IF NOT EXISTS idx_submission_challenge ON
submission(challenge_id);
CREATE INDEX IF NOT EXISTS idx_submission_team_correct ON
submission(team_id, is_correct);
CREATE INDEX IF NOT EXISTS idx_submission_user_challenge_time ON
submission(user_id, challenge_id, submitted_at);
CREATE INDEX IF NOT EXISTS idx_submission_challenge_correct ON
submission(challenge_id, is_correct);
CREATE INDEX IF NOT EXISTS idx_submission_submitted_at ON
submission(submitted_at);

-- Unique constraint to prevent duplicate solves per team
CREATE UNIQUE INDEX IF NOT EXISTS idx_submission_team_challenge_unique
ON submission(team_id, challenge_id)
WHERE is_correct = true;

-- Submission block indexes
CREATE INDEX IF NOT EXISTS idx_submission_block_user ON
submission_block(user_id);
CREATE INDEX IF NOT EXISTS idx_submission_block_challenge ON
submission_block(challenge_id);
CREATE INDEX IF NOT EXISTS idx_submission_block_until ON
submission_block(blocked_until);

-- Event config indexes
CREATE INDEX IF NOT EXISTS idx_event_config_status ON event_config(status);
CREATE INDEX IF NOT EXISTS idx_event_config_times ON
event_config(start_time, end_time);
```

```
-- Event rule indexes
CREATE INDEX IF NOT EXISTS idx_event_rule_version_number ON
event_rule_version(version_number);
CREATE INDEX IF NOT EXISTS idx_event_rule_version_created_by ON
event_rule_version(created_by);
CREATE INDEX IF NOT EXISTS idx_event_rule_version_created_at ON
event_rule_version(created_at);

-- Notification indexes
CREATE INDEX IF NOT EXISTS idx_notification_type ON notification(type);
CREATE INDEX IF NOT EXISTS idx_notification_published ON
notification(is_published);
CREATE INDEX IF NOT EXISTS idx_notification_published_created ON
notification(is_published, created_at);
CREATE INDEX IF NOT EXISTS idx_notification_created_at ON
notification(created_at);

-- Audit log indexes
CREATE INDEX IF NOT EXISTS idx_audit_log_user ON audit_log(user_id);
CREATE INDEX IF NOT EXISTS idx_audit_log_action ON audit_log(action);
CREATE INDEX IF NOT EXISTS idx_audit_log_resource_type ON
audit_log(resource_type);
CREATE INDEX IF NOT EXISTS idx_audit_log_resource ON
audit_log(resource_type, resource_id);
CREATE INDEX IF NOT EXISTS idx_audit_log_created_at ON
audit_log(created_at);
```

2. Team roles

- **Juan David Buitrago Salazar**

- Role: SCRUM Master and DevOps Practitioner (*Deployment and bug fixes*).



- **Luis David Garzón Morales**

- Role: Database administrator and backend developer.



- **Juan David Cardenas Galvis**

- Role: Backend and Frontend developer and testing



- **Alvaro Camilo Torres Rodríguez**

- Role: Backend developer and testing



- **David Felipe Chaparro Perez**
 - Role: Product Owner and developer



3. Github Repository

<https://github.com/Bellic12/RabbitCTF>